test_first/05/spec/system/add_project_spec.rb
```
require "rails_helper"
RSpec.describe "adding a project", type: :system do
  it "allows a user to create a project with tasks" do
    visit new_project_path
    fill_in "Name", with: "Project Runway"
    fill_in "Tasks", with: "Choose Fabric:3\nMake it Work:5"
    click_on("Create Project")
    visit projects_path
    @project = Project.find_by(name: "Project Runway")
    expect(page).to have_selector(
      "#project_#{@project.id} .name", text: "Project Runway")
    expect(page).to have_selector(
      "#project_#{@project.id} .total-size", text: "8")
  end
end
```

In the final two lines you're testing for the same text as you did in the previous example, but now you're forcing it to appear on a specific part of the page. You want each bit of text to be associated with a DOM ID representing the project item—using the Rails-blessed pattern <class>_<id> and then a DOM class representing type. This gives you a stronger test: no longer would a random 8 somewhere on the page cause a pass—now the 8 specifically has to be associated with the size of this project.

However, the test isn't completely brittle—nothing specifies, for example, that the elements are table rows and cells. So if you go off and redesign this page using more modern markup, as long as the size element is subordinate to the project element, the test will still pass. The view needs only minor changes to make this test pass:

test_first/05/app/views/projects/index.html.erb
```
<h1>All Projects</h1>
<table>
  <thead>
    <tr>
      <td>Project Name</td>
      <td>Total Project Size</td>
    </tr>
  </thead>
  <tbody>
    <% @projects.each do |project| %>
      <tr class="project-row" id="<%= dom_id(project) %>">
        <td class="name"><%= project.name %></td>
        <td class="total-size"><%= project.total_size %></td>
      </tr>
    <% end %>
  </tbody>
</table>
```